

Khulna University of Engineering & Technology

Department of Computer Science and Engineering

CSE 4102: Computer Graphics Laboratory

Final Project Report

Squid Game 3D

Interactive Computer Graphics Simulation & Two-Round Game

(Red Light, Green Light & The Glass Bridge)

A two-round 3D game made with C++ and modern OpenGL. Every object is built from basic shapes, lit by three kinds of light, Phong shaded and partly texture mapped.

Submitted by:

Arka Braja Prasad Nath
Roll: 2107055
Year / Term: 4th Year, 1st Term
Department of CSE, KUET

Submitted to:

Md Tajmilur Rahman
Lecturer, Dept. of CSE, KUET

Md. Mubtashim Abrar Nihal
Lecturer, Dept. of CSE, KUET

---

12 October 2026

Contents

1 Introduction **2**

1.1 Core Computer Graphics Concepts Implementation 2

2 The Game in Pictures **3**

2.1 Round 1: Red Light, Green Light 3

2.2 Round 2: The Glass Bridge 4

3 Tools Used **4**

4 How to Play **5**

5 Program Structure **5**

6 Basic Shapes **6**

7 Transformations **7**

7.1 Multi-View Orthographic and Isometric Projections 7

8 Objects and the Shapes They Are Made Of **8**

8.1 Player 456 and the other contestants 9

8.2 Pink guard (Round 1) 10

8.3 Young-hee, the giant doll 11

8.4 Young-hee doll head detail 12

8.5 The dead tree 13

8.6 Village house 14

8.7 Countdown clock 15

8.8 The glass bridge 16

8.9 Glass bridge start platform 17

8.10 Round 2 bridge guard 18

8.11 Light fixtures 19

8.12 Round 1 arena environment 20

8.13 Round 2 glass bridge hall environment 21

9 Lighting **22**

10 Phong Shading **24**

11 Texture Mapping **25**

12 Animation and Game Logic **26**

13 Cameras and Viewport Pipeline **26**

13.1 Camera Architecture and Mathematical Mechanism 26

13.2 Screen Dimensions, Aspect Ratio, and Viewport Calculation 27

13.3 Smooth Camera Easing and Interactive Orbiting 28

13.4 Camera Perspectives and Multi-Pass CCTV Rendering 28

14 Conclusion **29**

1 Introduction

This project presents an interactive 3D Computer Graphics game inspired by the TV series *Squid Game*, developed using modern OpenGL 3.3 Core Profile and C++17. The project showcases fundamental computer graphics techniques including hierarchical object modeling, 3D geometric transformations, interactive camera controls, Phong illumination with three distinct light types, texture mapping, and real-time animation physics.

The game consists of two complete, playable rounds:

- **Round 1 – Red Light, Green Light:** The player must sprint across a vast desert arena to the finish line within 60 seconds while the giant Young-hee doll sings. Movement is allowed only during Green Light; moving when the doll turns to scan during Red Light triggers immediate elimination by sniper guards with particle blood effects.
- **Round 2 – The Glass Bridge:** The player must cross 16 pairs of elevated glass stepping stones suspended in a neon-lit industrial hall. For each step, one pane is tempered glass and holds weight, while the other pane shatters into 55 physics shards upon impact, dropping the player. Surviving contestants navigate forward together.

1.1 Core Computer Graphics Concepts Implementation

The implementation adheres to the curriculum requirements of CSE 4102 by translating core theoretical graphics principles into functional code:

Core CG Concept	Implementation in Project	Reference
Hierarchical Modeling	Scene graph composition via `glPushMatrix()`/`glPopMatrix()`; parent-child inheritance (limbs, fingers, doll head, gantry)	Section 6, 8
3D Transformations	Translation (motion, offsets), Rotation (joint swings, steering, scanning), Scaling (primitive deformation, doll vs player scale)	Section 12
Lighting Models	Directional (sun / ceiling array), Point (floodlight tower / chandelier / wall lamps), Spot (doll eye searchlight / tracking spotlight)	Section 9
Phong Shading	Ambient, diffuse, and Blinn-Phong specular reflections with tunable shininess exponents computed per-pixel in GLSL fragment shaders	Section 10
Texture Mapping	Procedural and image-based texture mapping with repeating UV coordinates for sand ground, tree bark, and concrete walls	Section 11
Camera Projections	7 dynamic camera modes (3rd person, overhead, top-down, doll close-up, free orbit) using perspective projection and view matrices	Section 13
Physics & Animation	Trigonometric walk cycle animations, smooth head easing, gravity fall physics, glass fracture shards, and blood particle emitters	Section 12

Table 1: Mapping of core computer graphics concepts to technical implementations in the project.

All screenshots and visual assets in this report are captured directly from the real-time OpenGL rendering engine.

2 The Game in Pictures

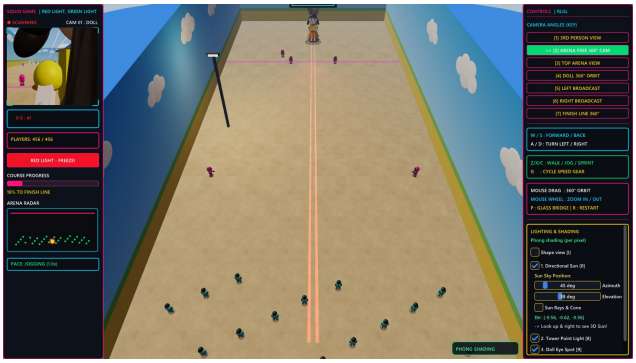
2.1 Round 1: Red Light, Green Light



Figure 1: Start screen. Left panel: live doll camera, timer, player count, progress bar and a mini radar. Right panel: camera buttons, controls and the lighting panel.



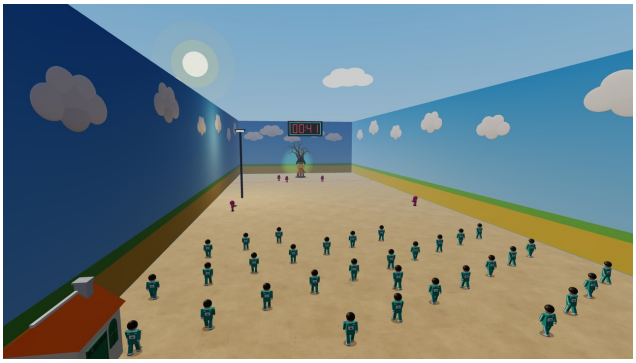
(a) Green light – everyone walks.



(b) Red light – the doll turns and scans with red lasers.



(c) Player 456 crossed the finish line.



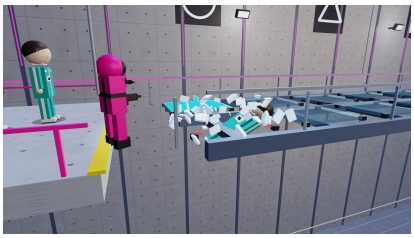
(d) The whole arena, seen from behind the start.

Figure 2: Round 1 while playing.

2.2 Round 2: The Glass Bridge



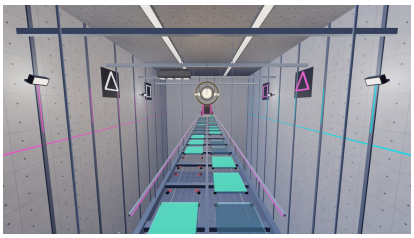
Figure 3: Player 456's turn. Green panes are proven safe, the red marks show broken panes. The cyan and pink panes in front are the two choices: press `A` for left or `D` for right.



(a) Pane shatter physics



(b) Player 456 victory



(c) Light hall architecture

Figure 4: Round 2 in-game moments and environment.

3 Tools Used

Item	What we used
Language	C++17, compiled with MinGW-w64 g++
Graphics	OpenGL 3.3 Core Profile with GLSL 330 shaders
Window and input	GLFW 3
OpenGL loader	GLAD
Math	GLM (vectors and matrices)
User interface	Dear ImGui (side panels, buttons, overlays)
Images	stb_image (loads the PNG textures)
Sound	Windows multimedia API (song + generated sound effects)
3D models	None imported – every object is built in code from basic shapes

4 How to Play

Key	Action
Both rounds	
`P`	Switch between Round 1 and Round 2
`R`	Restart the round
`0 / 8 / 9`	Turn the directional / point / spot light on or off
`I`	Shape view: color every part by its basic shape
`Mouse drag, wheel`	Orbit the camera, zoom
Round 1	
`W / S`	Walk forward / backward
`A / D, Q / E`	Turn left / right, step sideways
`Z / X / C (or Shift)`	Walk / jog / sprint
`1 -- 7`	Camera modes
Round 2	
`A / D`	Jump to the left / right pane
`S`	Skip to Player 456's turn
`Tab`	Fast-forward the other players
`1 -- 5`	Camera modes
`7`	Toggle wall lamps

5 Program Structure

The code is split into small folders, one job each:

Folder	What it does
`main_modern.cpp`	Main loop, keyboard/mouse, lights setup, user interface, capture mode
`engine/`	OpenGL building blocks: Shader, VAO, VBO, EBO, Mesh, Texture, basic shapes, matrix stack
`shaders/`	`default.vert` and `default.frag` (Phong lighting)
`render/`	Light fixture models, the three textures
`contestants/, guards/, doll/`	The characters
`environment/`	Round 1 arena: sky, sand, walls, houses, clock
`glassbridge/`	Round 2: bridge, hall, platforms, glass shards, cameras, game rules
`camera/, audio/`	Round 1 cameras, sound
`core/Config.h`	All colors, sizes, speeds and timers in one place
`tools/`	Shape showcase (report figures), texture generator, figure script

Every frame the program does the same five steps:

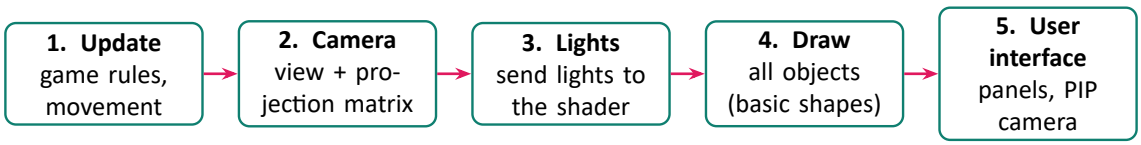


Figure 5: What happens in one frame (about 60 times per second).

World size. All sizes are in game units. Round 1 arena: x from -24 to 24, z from -75 to 75, walls 22 high. Start line at $z = -52$, finish line at $z = 45$, doll at $(0, 0, 58)$, 2.6 times bigger than normal. Round 2: 16 glass steps, the first at $z = 3$, one step every 3.3 units, at height $y = 16$.

6 Basic Shapes

The whole game is built from only seven basic shapes. Figure 6 shows each one, rendered by the program itself. The thin dark lines show the triangles the GPU actually draws.

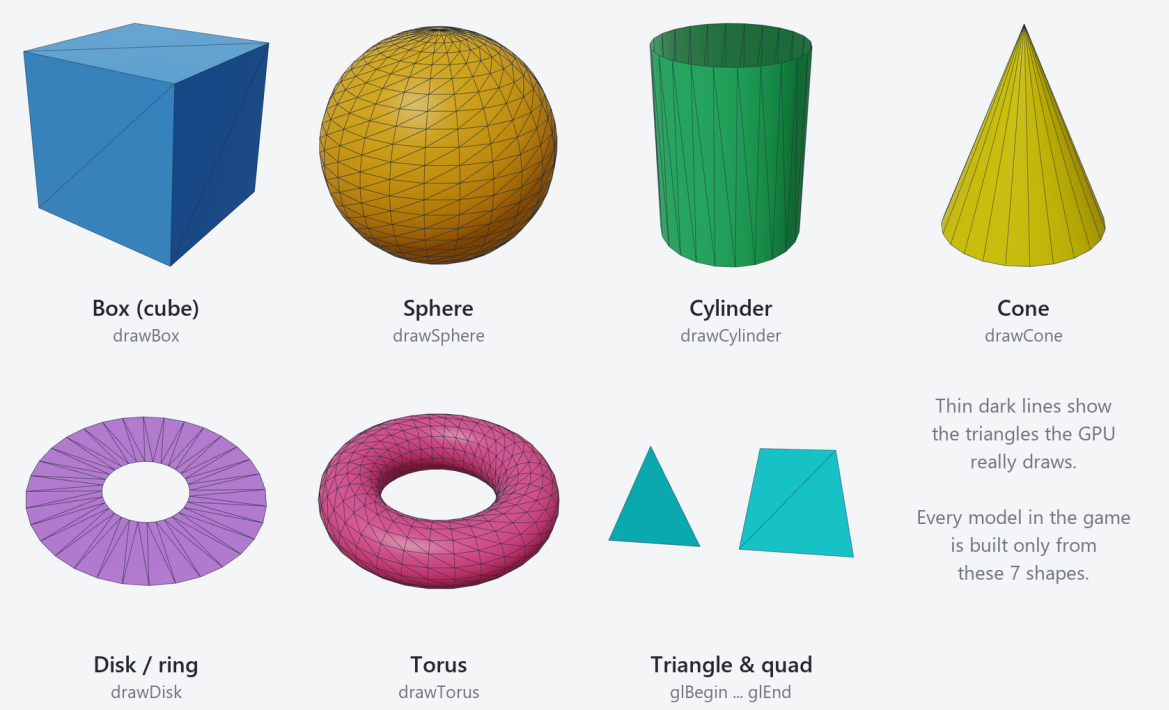


Figure 6: The seven basic shapes and the function that draws each one.

Shape	Settings	How it is made	Used for (examples)
Box	width, height, depth	6 faces × 4 corners = 24 vertices, 12 triangles	bodies, legs, walls, glass panes, rails
Sphere	radius	24 rings × 24 slices = 1152 triangles	heads, eyes, hands, clouds, bulbs
Cylinder	bottom radius, top radius, height	a ring of side rectangles (no lids)	tree branches, neck, cables
Cone	radius, height	a cylinder whose top radius is 0	(cone beams)
Disk	inner radius, outer radius	flat ring of triangles	doll's collar, lens, sleeve cuffs
Torus	tube radius, ring radius	a tube bent into a ring	chandelier rings, circle symbols
Polygon	any corner points	triangles / quads typed in by hand	roofs, sand floor, skirt, sky

Table 2: The basic shapes. A box is always made as a $1 \times 1 \times 1$ cube and stretched with *scale*; a sphere is always radius 1 and scaled to the wanted size.

7 Transformations

Each shape is moved into place with three transformations. In 3D we use 4×4 matrices:

$$T = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad S = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad R_y(\theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- **Translate** (T / `glTranslatef`) moves a part or character in 3D world or local coordinate space.
- **Rotate** (R / `glRotatef`) turns a part or body joint (e.g., pendulum limb swing, doll head $180°$ scan).
- **Scale** (S / `glScalef`) sizes primitives: deforms unit boxes and spheres into anatomical body parts (thin soles, flat walls, oval blushes) and scales the colossal doll ($2.5\times$) relative to normal contestants.
- **Hierarchical Modeling** (`glPushMatrix` / `glPopMatrix`): A part's final matrix is parent $\times$ T $\times$ R $\times$ S. Child parts inherit parent transformations (the hand follows the arm, and the arm follows the body). `glPushMatrix()` preserves the parent matrix and `glPopMatrix()` restores it, isolating local changes from adjacent objects.

Example – one leg of Player 456 (from `Contestants.cpp`):

```
glPushMatrix();
glTranslatef(side * 0.15f, 0.62f, 0.0f); // move to the hip joint
glRotatef(legAngle, 1.0f, 0.0f, 0.0f); // swing the leg forward/back (walk)
glTranslatef(0.0f, -0.279f, 0.0f); // the leg hangs below the hip
glScalef(0.18f, 0.558f, 0.18f); // stretch the unit cube into a leg
glutSolidCube(1.0);
glPopMatrix();
```

Note: these old-style function names are kept on purpose, but they run on modern OpenGL. Our `GLCompat` layer keeps its own matrix stack (GLM) and draws the shape with VAO/VBO buffers and shaders.

7.1 Multi-View Orthographic and Isometric Projections

In engineering computer graphics and 3D geometric modeling, standard multi-view projections are employed to inspect and verify 3D objects across principal coordinate axes without perspective distortion:

- **Front View (Elevation):** Projection along the principal viewing axis (xy-plane) displaying the frontal silhouette, facial features, and vertical proportion hierarchy.
- **Side View (Profile):** Projection along the orthogonal lateral axis (yz-plane) capturing depth profile, limb swing stance, and front-to-back extents.
- **Top View (Plan):** Projection down the vertical y-axis (xz-plane) displaying structural footprint, horizontal symmetry, and planar orientations.
- **Isometric View:** An axonometric 3D projection where coordinate axes are equally foreshortened, revealing volumetric depth and spatial relationships ($R_x\, 35.264° \cdot R_y\, 45°$).

$$P_{\text{ortho}} = \begin{bmatrix} \frac{1}{r} & 0 & 0 & 0 \\ 0 & \frac{1}{t} & 0 & 0 \\ 0 & 0 & -\frac{2}{f-n} & -\frac{f+n}{f-n} \\ 0 & 0 & 0 & 1 \end{bmatrix} \qquad R_{\text{iso}} = R_x(35.264°) \cdot R_y(45°)$$

Canonical views are rendered with `glm::ortho(-w, w, -h, h, 0.1f, 100.0f)` and `glm::lookAt(eye, center, up)`.

8 Objects and the Shapes They Are Made Of

Every 3D model in the project is procedurally assembled from the seven primitive shapes introduced in Section 6, without importing third-party mesh files. For each object, we provide a complete technical specification sheet across three core dimensions:

1. **Canonical Multi-View Projections:** High-resolution orthogonal elevations and axonometric view displaying **(a) Front (elevation)**, **(b) Side (profile)**, **(c) Top (plan)**, and **(d) Isometric** aspects.
2. **Hierarchical Shape Decomposition:** The finished textured model rendered alongside its exploded primitive view, color-coded by shape category: Box Sphere Cylinder Cone Disk Torus Polygon.
3. **Mathematical Transformation Hierarchy:** Exact local translations (T), rotations (R), scales (S), and primitive dimensions.

Coordinate Conventions and Hierarchy. All local coordinate descriptions use a right-handed Cartesian system: $+x$ points right, $+y$ points vertically upward, and $+z$ points forward along the principal facing direction. Child parts inherit their parent's coordinate frame via the matrix stack (`glPushMatrix()` / `glPopMatrix()`). The notation "$\pm$" designates symmetric left/right pairs. Rotations indicate axis and angle (e.g., $R_x\,25^\circ$).

Shape Primitive	Color Code	Parametric Generation / Topology
Box	Cyan	12 triangles from unit cube, scaled by (s_x, s_y, s_z)
Sphere	Orange	UV sphere (24×24 rings), polar coordinates (θ, ϕ)
Cylinder	Blue	Longitudinal triangle strip with taper $(r_{\text{base}} \to r_{\text{top}})$
Cone	Green	Degenerate tapered cylinder ($r_{\text{top}} = 0$, apex vertex)
Disk	Yellow	Planar annular ring ($r_{\text{inner}} \le r \le r_{\text{outer}}$)
Torus	Magenta	Torus swept by ring radius R and tube radius r
Polygon	Red	Planar vertex arrays (roofs, sand ground, sky quads)

Table 3: Primitive shapes and color assignments used in all exploded decomposition figures.

Object	Box	Sphere	Cyl.	Disk	Torus	Poly.	Line	Total
Player 456 / contestant	49	4						53
Pink guard (Round 1)	19	4						23
Bridge guard (Round 2)	9	2			1			12
Young-hee doll (body)	40	30	3	5		18		96
Young-hee doll head	10	40	1					51
Dead tree		157	156					313
Village house	21	1			3	2	2	29
Countdown clock	55							55
Glass bridge (3 steps shown)	48							48
Start platform	17							17
Light fixtures (5 kinds)	21	9	1	1	2			34

Table 4: Number of parts in each object, counted by the program while drawing it.

Documentation Structure. The following sheets detail each 3D asset individually on a dedicated page, verifying its geometric symmetry, proportions, and procedural transformation pipeline.

8.1 Player 456 and the other contestants

The same model is used for every contestant in both rounds; only the number on the chest and back changes. **Placed in the world with:** $T(x, 0, z) \cdot R_y(\text{facing}) \cdot S(1.15)$ (other contestants $S(1.10)$).

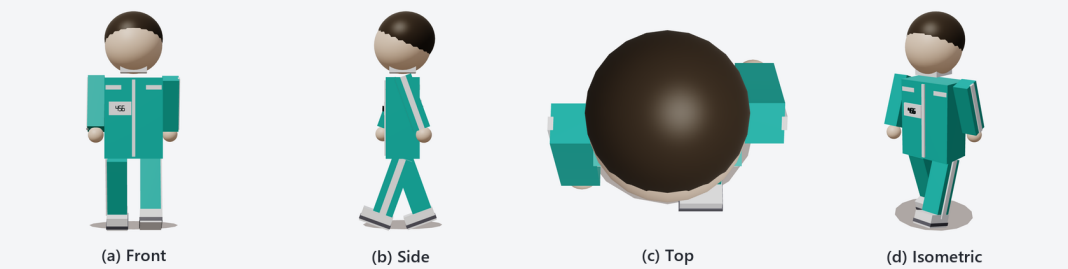


Figure 7: Player 456 multi-view projections: (a) Front, (b) Side profile, (c) Top view, (d) Isometric 3D view.

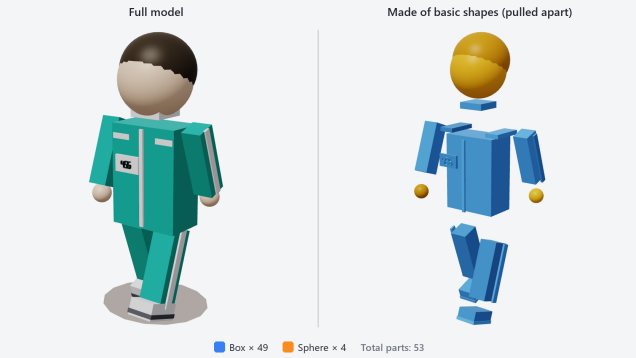


Figure 8: Player 456 shape decomposition: 49 boxes and 4 spheres.

Part	Shape	Scale / size	Translate (position)	Rotate
Legs $\times 2$	Box	$0.18 \times 0.56 \times 0.18$	$(\pm 0.15,\ 0.34,\ 0)$	R_x swing at hip (y=0.62)
Leg stripes $\times 2$	Box	$0.02 \times 0.56 \times 0.06$	$(\pm 0.245,\ 0.34,\ 0)$	moves with leg
Shoes $\times 2$	Box	$0.19 \times 0.12 \times 0.28$	$(\pm 0.15,\ 0.04,\ 0.04)$	moves with leg
Shoe soles $\times 2$	Box	$0.20 \times 0.03 \times 0.29$	$(\pm 0.15,\ 0.01,\ 0.04)$	moves with leg
Jacket (body)	Box	$0.52 \times 0.72 \times 0.30$	$(0,\ 0.98,\ 0)$	–
Zipper	Box	$0.03 \times 0.68 \times 0.02$	$(0,\ 0.98,\ 0.16)$	–
Shoulder stripes $\times 2$	Box	$0.14 \times 0.04 \times 0.31$	$(\pm 0.18,\ 1.25,\ 0)$	–
Chest number patch	Box	$0.20 \times 0.12 \times 0.02$	$(-0.12,\ 1.07,\ 0.16)$	–
Back number patch	Box	$0.32 \times 0.18 \times 0.02$	$(0,\ 1.05,\ -0.16)$	$R_y\ 180°$
Number “456” (front + back)	Box $\times 30$	7-segment bars, 0.021×0.005 (front)	on the patches	–
Arms $\times 2$	Box	$0.15 \times 0.49 \times 0.15$	$(\pm 0.335,\ 1.10,\ 0)$	R_x swing at shoulder (y=1.34)
Arm stripes $\times 2$	Box	$0.02 \times 0.49 \times 0.05$	$(\pm 0.41,\ 1.10,\ 0)$	moves with arm
Hands $\times 2$	Sphere	radius 0.07	$(\pm 0.335,\ 0.80,\ 0)$	moves with arm
Collar	Box	$0.24 \times 0.06 \times 0.24$	$(0,\ 1.42,\ 0)$	–
Face	Sphere	radius 0.26	$(0,\ 1.64,\ 0)$	–
Hair	Sphere	radius 0.26, $S(1.02, 0.95, 1.02)$	$(0,\ 1.70,\ -0.03)$	–

8.2 Pink guard (Round 1)

Five guards stand around the arena. The mask symbol shows the rank: circle, triangle or square. **Placed with:** $T(x, 0, z) \cdot R_y(\text{facing}) \cdot S(1.2)$. When a guard shoots, it turns to the target: facing $=$ atan2$(x_t - x_g,\ z_t - z_g)$.

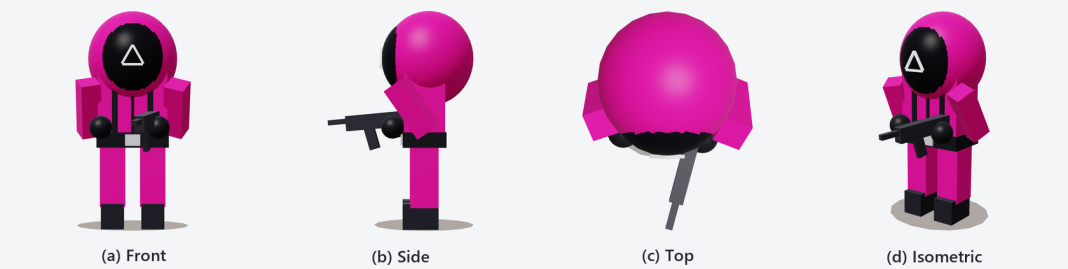


Figure 9: Pink guard multi-view projections: (a) Front, (b) Side profile, (c) Top view, (d) Isometric 3D view.

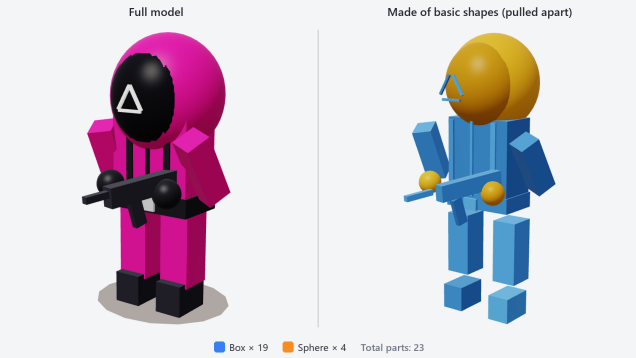


Figure 10: Triangle guard shape decomposition: 19 boxes and 4 spheres.

Part	Shape	Scale / size	Translate (position)	Rotate
Boots $\times 2$	Box	$0.18 \times 0.20 \times 0.28$	$(\pm 0.16,\ 0.10,\ 0.03)$	–
Legs $\times 2$	Box	$0.20 \times 0.50 \times 0.22$	$(\pm 0.16,\ 0.42,\ 0)$	–
Body	Box	$0.55 \times 0.60 \times 0.32$	$(0,\ 0.95,\ 0)$	–
Zipper	Box	$0.024 \times 0.58 \times 0.015$	$(0,\ 0.95,\ 0.165)$	–
Belt + buckle	Box $\times 2$	$0.57 \times 0.12 \times 0.34$; $0.12 \times 0.09 \times 0.025$	$(0,\ 0.70,\ 0)$; $(0,\ 0.70,\ 0.175)$	–
Shoulder straps $\times 2$	Box	$0.045 \times 0.50 \times 0.015$	$(\pm 0.14,\ 0.98,\ 0.163)$	–
Neck collar	Box	$0.34 \times 0.09 \times 0.32$	$(0,\ 1.22,\ 0)$	–
Left / right arm	Box	$0.16 \times 0.45 \times 0.16$	$(-0.35,\ 0.95,\ 0.08)$ / $(0.35,\ 0.95,\ 0.10)$	$R_x 25^\circ R_y 20^\circ$ / $R_x 35^\circ R_y -25^\circ$
Gloves $\times 2$	Sphere	radius 0.09	$(-0.25,\ 0.80,\ 0.22)$, $(0.20,\ 0.80,\ 0.25)$	–
Hood	Sphere	$S(0.36, 0.38, 0.36)$	$(0,\ 1.35,\ -0.04)$	–
Black mask	Sphere	$S(0.30, 0.32, 0.28)$	$(0,\ 1.35,\ 0.06)$	–
Triangle symbol	Box $\times 3$	base 0.16×0.018, sides 0.018×0.165	mask $+(0, 0, 0.285)$; sides at $x = \pm 0.042$	sides $R_z \mp 30^\circ$
Gun body	Box	$0.08 \times 0.12 \times 0.55$	$(0.12,\ 0.85,\ 0.35)$	$R_y -15^\circ\ R_x 5^\circ$
Barrel, magazine	Box $\times 2$	$0.04 \times 0.04 \times 0.22$; $0.04 \times 0.20 \times 0.08$	gun $+(0, 0.02, 0.32)$; gun $+(0, -0.12, 0.08)$	magazine $R_x 12^\circ$
Muzzle flash (only when shooting)	Sphere	radius 0.14	gun $+(0,\ 0.02,\ 0.48)$	–

Circle guards use one Torus (tube 0.016, ring 0.088) and square guards use four boxes instead of the triangle.

8.3 Young-hee, the giant doll

The doll stands with her arms around the tree. Only her head turns. **Placed with:** $T(0, 0, 58) \cdot S(2.6)$; the head adds $T(0, 2.50, 0) \cdot R_y(\text{headAngle})$.

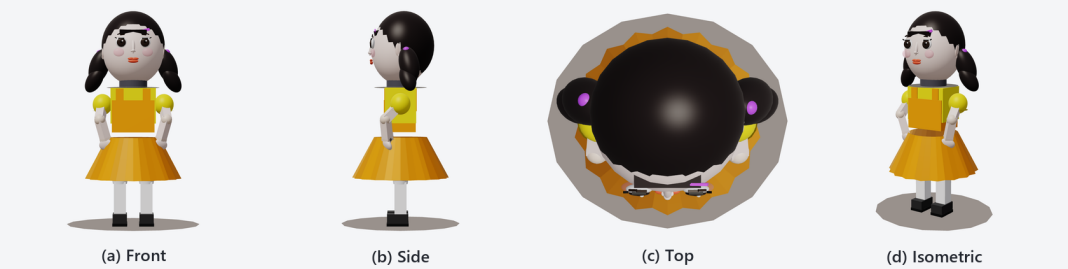


Figure 11: Young-hee giant doll multi-view projections: (a) Front, (b) Side profile, (c) Top view, (d) Isometric 3D view.

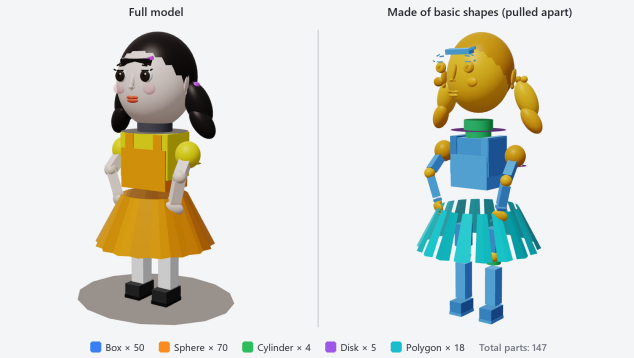


Figure 12: The doll body shape decomposition: 96 parts. The skirt is 18 hand-made polygon pleats.

Part	Shape	Scale / size	Translate (position)	Rotate
Neck	Cylinder	radius 0.18, height 0.28	$(0,\ 2.05,\ 0)$	$R_x\ 90°$
White collar	Disk	radius $0.18 \rightarrow 0.38$	$(0,\ 1.96,\ 0)$	$R_x\ 90°$
Yellow shirt	Box	$0.64 \times 0.50 \times 0.44$	$(0,\ 1.70,\ 0)$	–
Orange dress top	Box	$0.60 \times 0.45 \times 0.45$	$(0,\ 1.55,\ 0.01)$	–
Dress straps $\times 2$	Box	$0.11 \times 0.32 \times 0.46$	$(\pm 0.19,\ 1.78,\ 0)$	–
Pleated skirt	Polygon $\times 18$	top radius 0.38, bottom 0.65, height 0.58	$(0,\ 1.25,\ 0)$	pleat i at $i \times 20°$
Sleeves $\times 2$	Sphere + Disk	$S(0.14, 0.15, 0.14)$; ring $0.08 \rightarrow 0.135$	shoulder $(\pm 0.44,\ 1.72,\ 0)$	cuff $R_x\ 90°$
Upper arms $\times 2$	Box	$0.12 \times 0.30 \times 0.12$	shoulder $+(0, -0.15, 0)$	$R_x -46° R_y \pm 16° R_z \pm 6°$
Elbows $\times 2$	Sphere + Box	radius 0.078; ring 0.082×0.02	shoulder $+(0, -0.30, 0)$	follows upper arm
Forearms $\times 2$	Box	$0.115 \times 0.29 \times 0.115$	elbow $+(0, -0.15, 0)$	$R_x 58° R_y \pm 34° R_z \mp 8°$
Wrists $\times 2$	Box + Sphere + Disk	cuff 0.12×0.035; ball $r\ 0.052$; hole $r\ 0.024$	elbow $+(0, -0.29, 0)$	hole $R_y \mp 90°$
Palms $\times 2$	Sphere	$S(0.11, 0.085, 0.045)$	wrist $+(0, -0.08, 0.015)$	$R_y \mp 24° R_x 12°$
Fingers $\times 8$, thumbs $\times 2$	2 Sphere + 2 Box each	joints $r\ 0.010$–0.014	finger f at $x = -0.038 + 0.025f$	curl $R_x 16°$
Thighs $\times 2$	Box	$0.18 \times 0.32 \times 0.18$	$(\pm 0.19,\ 0.85,\ 0)$	–
Knees $\times 2$	Sphere	radius 0.09	$(\pm 0.19,\ 0.68,\ 0)$	–
Sock tops $\times 2$	Cylinder	radius $0.11 \rightarrow 0.10$, height 0.05	$(\pm 0.19,\ 0.64,\ 0)$	$R_x\ 90°$
Socks $\times 2$	Box	$0.17 \times 0.52 \times 0.17$	$(\pm 0.19,\ 0.36,\ 0)$	–
Shoes + soles $\times 2$	Box $\times 2$	$0.20 \times 0.16 \times 0.30$; $0.21 \times 0.04 \times 0.31$	$(\pm 0.19,\ 0.09,\ 0.04)$; y=0.02	–

8.4 Young-hee doll head detail

The head rotates independently by $\pm 180^\circ$ for player scanning and features dual ocular laser spotlights. **Placed with:** local offset $T(0, 2.50, 0) \cdot R_y(\text{headAngle})$ relative to the torso.



Figure 13: Doll head multi-view projections: (a) Front, (b) Side profile, (c) Top view, (d) Isometric 3D view.

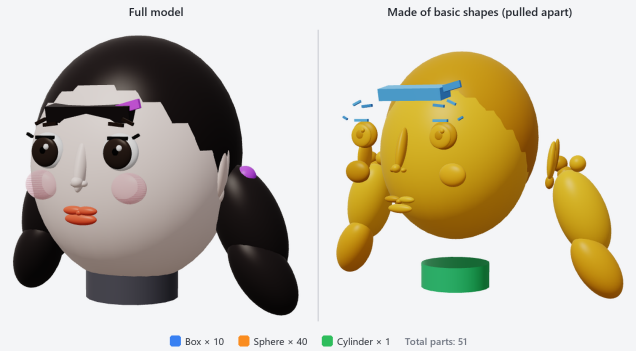


Figure 14: The doll's head on its own: 51 parts, mostly squashed spheres.

Part	Shape	Scale / size	Translate (position)	Rotate
Neck joint ring	Cylinder	radius 0.20, height 0.14	$(0,\ -0.42,\ 0)$	$R_x\ 90^\circ$
Face	Sphere	$S(0.44, 0.50, 0.42)$	$(0,\ 0,\ 0)$	–
Nose bridge, tip	Sphere $\times 2$	$S(0.026, 0.115, 0.034)$; $S(0.024, 0.022, 0.024)$	$(0, 0.03, 0.408)$; $(0, -0.04, 0.444)$	bridge $R_x -18^\circ$
Nostrils $\times 2$	Sphere	$S(0.014, 0.012, 0.016)$	$(\pm 0.023,\ -0.046,\ 0.430)$	–
Lips (4 parts)	Sphere $\times 4$	upper $S(0.046, 0.018, 0.019)$, lower $S(0.072, 0.024, 0.022)$	$(\pm 0.030, -0.152, 0.422)$, $(0, -0.185, 0.418)$	–
Blush $\times 2$ (32% see-through)	Sphere	$S(0.075, 0.060, 0.034)$	$(\pm 0.20,\ -0.06,\ 0.392)$	–
Eyes $\times 2$	4 Sphere + 2 Box each	white $S(0.078, 0.072, 0.030)$, iris, pupil, shine, lashes	$(\pm 0.165,\ 0.078,\ 0.395)$	lash wing $R_z \pm 28^\circ$
Eyebrows $\times 2$	Box $\times 2$ each	0.062×0.014; 0.050×0.012	$(\pm 0.150, 0.182, 0.402)$; $(\pm 0.198, 0.193, 0.397)$	$R_z \mp 6^\circ$; $R_z \mp 24^\circ$
Hair	Sphere	$S(0.455, 0.495, 0.435)$	$(0,\ 0.06,\ -0.05)$	–
Fringe (bangs)	Box	$0.38 \times 0.065 \times 0.11$	$(0,\ 0.23,\ 0.32)$	$R_x 12^\circ$
Hair clip	Box	$0.10 \times 0.035 \times 0.016$	$(0.18,\ 0.25,\ 0.36)$	$R_z 14^\circ$
Ears $\times 2$	Sphere $\times 4$ each	main $S(0.024, 0.105, 0.055)$	$(\pm 0.42,\ -0.02,\ 0.02)$	$R_y \pm 14^\circ R_z -6^\circ$
Pigtails $\times 2$	Sphere $\times 6$ each	4 beads $r\,0.046$; hair $S(0.14, 0.26, 0.14)$, $S(0.11, 0.20, 0.11)$	$(\pm 0.44,\ -0.07,\ -0.10)$	$R_z \pm 20^\circ$, $R_z \pm 32^\circ$
Red-light lasers $\times 2$	Cylinder	radius $0.010 \rightarrow 0.07$, length 34	from each eye, along $+z$	follows the head

8.5 The dead tree

The tree is made by a small *recursive* function: a branch draws itself, then two smaller branches at its tip, and so on. Every branch is a tapered cylinder with a sphere "knuckle" at the end, so joints have no gaps. The bark is a texture (Section 11). **Placed with:** $T(0, 0, 59.85)$, just behind the doll.

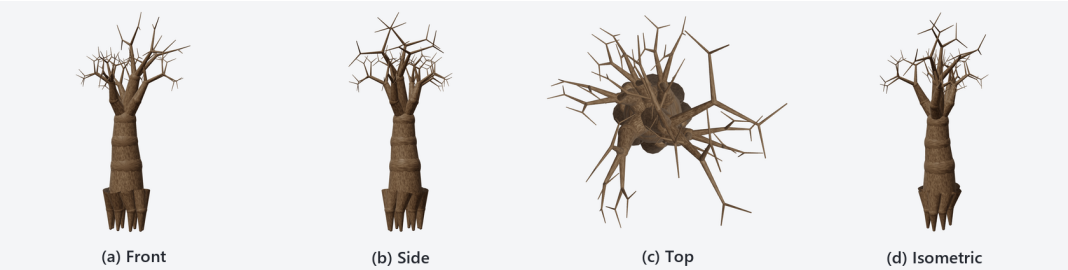


Figure 15: Dead tree multi-view projections: (a) Front, (b) Side profile, (c) Top view, (d) Isometric 3D view.

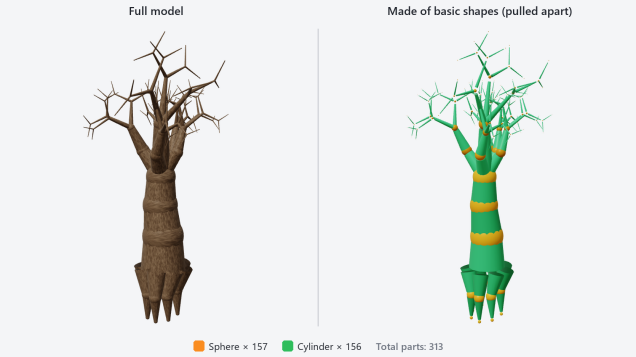


Figure 16: The tree shape decomposition: 156 cylinders and 157 spheres.

Part	Shape	Scale / size	Translate (position)	Rotate
Roots $\times 8$	2 Cylinder + 2 Sphere each	radius $0.65 \to 0.38 \to 0.12$, length $1.6 + 1.6$	$(1.3\cos\theta,\ 0.25,\ 1.3\sin\theta)$, $\theta = i \cdot 45^\circ$	$R_y\theta\ R_x 75^\circ$, then $R_x 18^\circ$
Trunk part 1	Cylinder	radius $1.45 \to 1.25$, height 2.2	$(0,\ 0,\ 0)$	$R_x -90^\circ$ (stand up)
Trunk part 2	Cylinder	radius $1.25 \to 1.05$, height 2.0	on top of part 1	$R_x 3.5^\circ R_y -4^\circ$
Trunk part 3	Cylinder	radius $1.05 \to 0.88$, height 1.8	on top of part 2	$R_x -4.5^\circ R_y 5^\circ$
Fork knuckle	Sphere	radius 0.92	top of trunk (≈ 6 high)	–
Big branches $\times 5$	2 Cylinder + 3 twigs each	base radius 0.50–0.56, length 3.2–3.6	at the fork	R_z: $-48^\circ, 48^\circ, -135^\circ, 135^\circ, 180^\circ$; R_x: 28°–38°
Top spire	Cylinder + 3 twigs	radius $0.65 \to 0.42$, length 3.5	at the fork	$R_x 8^\circ R_y 12^\circ$
Twigs (recursive)	Cylinder + Sphere	each level: length $\times 0.70$, radius $\times 0.45$	at the parent's tip	$R_y \pm (26^\circ$–$40^\circ)$, $R_x \pm (35^\circ$–$70^\circ)$

8.6 Village house

One house model is drawn 9 times: five in a row behind the start ($z = -67$) and two on each side wall ($x = \pm 22$, turned $\pm 90°$). **Placed with:** $T(x, 0, z) \cdot R_y(\text{angle})$.

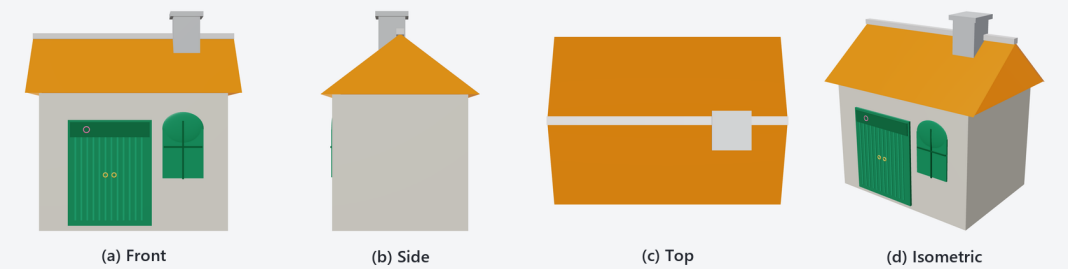


Figure 17: Village house multi-view projections: (a) Front, (b) Side profile, (c) Top view, (d) Isometric 3D view.

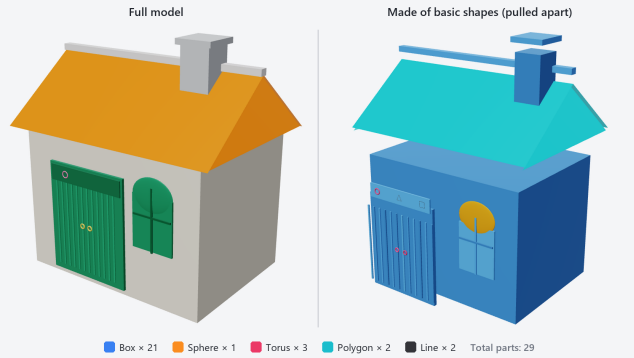


Figure 18: Village house shape decomposition: 29 parts. The roof is made of hand-made triangles and quads.

Part	Shape	Scale / size	Translate (position)	Rotate
Walls (body)	Box	$4.4 \times 3.2 \times 3.2$	$(0,\ 1.6,\ 0)$	–
Roof ends $\times 2$	Polygon (triangle)	base 3.7, height 1.4	$x = \pm 2.45$, from $y = 3.2$	normal $(\pm 0.71, 0.71, 0)$
Roof sides $\times 2$	Polygon (quad)	5.0 long, 1.85 deep, 1.4 rise	front / back, from $y = 3.2$	normal $(0, 0.71, \pm 0.71)$
Roof ridge	Box	$5.0 \times 0.14 \times 0.18$	$(0,\ 4.65,\ 0)$	–
Chimney + cap	Box $\times 2$	$0.65 \times 1.4 \times 0.65$; $0.80 \times 0.12 \times 0.80$	$(1.3,\ 4.4,\ 0.2)$; $(1.3,\ 5.15,\ 0.2)$	–
Door frame	Box	$1.95 \times 2.45 \times 0.08$	$(-0.55,\ 1.35,\ 1.62)$	–
Door bars $\times 12$	Box	$0.04 \times 2.0 \times 0.03$	$x = -1.35$ to 0.19, every 0.14	–
Sign above door	Box	$1.85 \times 0.35 \times 0.04$	$(-0.55,\ 2.35,\ 1.66)$	–
Symbols ◯ △ □	Torus + 2 line loops	ring 0.07; triangle/square 0.14 wide	on the sign	–
Door rings $\times 2$	Torus	tube 0.015, ring 0.05	$(-0.65,\ 1.30,\ 1.68)$, $(-0.45,\ 1.30,\ 1.68)$	–
Window + round top	Box + Sphere	$0.95 \times 1.15 \times 0.04$; $S(0.48, 0.38, 0.04)$	$(1.15,\ 1.8,\ 1.64)$; $+(0, 0.58, 0)$	–
Window bars $\times 2$	Box	0.04×1.35; 0.90×0.04	window $+(0, 0.08, 0.03)$, $+(0, 0.15, 0.03)$	–

8.7 Countdown clock

A digital clock hangs above the finish line and shows the remaining game time on both sides. Each digit is formed using 7-segment digital bars, updated synchronously each second. **Placed with:** $T(0, 17.5, 42) \cdot S(2)$.

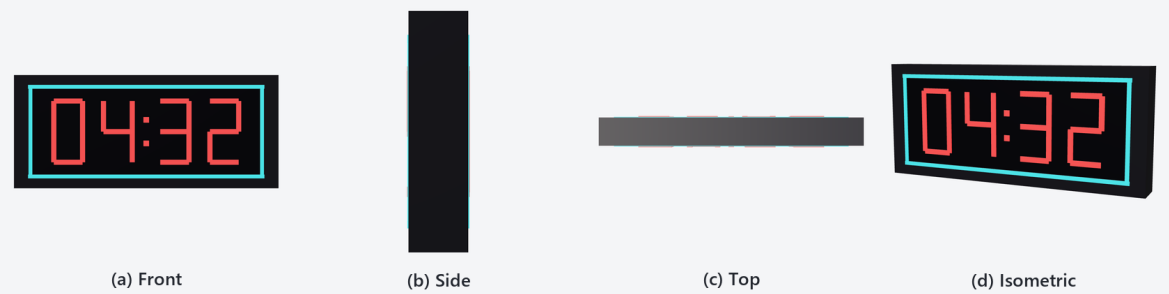


Figure 19: Countdown scoreboard clock multi-view projections: (a) Front, (b) Side profile, (c) Top view, (d) Isometric 3D view.

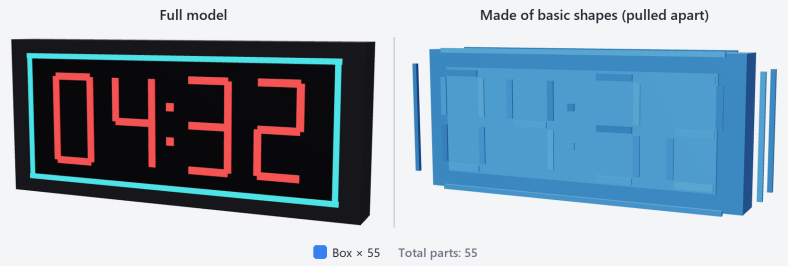


Figure 20: Clock showing 04:32: 55 boxes.

Part	Shape	Scale / size	Translate (position)	Rotate
Outer frame	Box	$5.6 \times 2.4 \times 0.6$	$(0,\ 0,\ 0)$	–
Glowing border $\times 4$ (each side)	Box	$5.0 \times 0.08 \times 0.06$, $0.08 \times 1.9 \times 0.06$	$y = \pm 0.95$; $x = \pm 2.45$; $z = \pm 0.29$	–
Black screen (each side)	Box	$4.8 \times 1.8 \times 0.04$	$(0,\ 0,\ \pm 0.29)$	–
Digit bars (each digit)	Box $\times$ up to 7	across 0.61×0.11, up 0.11×0.65	digits at $x = \pm 1.65,\ \pm 0.60$	back side mirrored
Colon dots $\times 2$ (each side)	Box	$0.12 \times 0.12 \times 0.02$	$(0,\ \pm 0.25,\ \pm 0.33)$	–

Seven-segment bitmasks: A (top), B (top-right), C (bottom-right), D (bottom), E (bottom-left), F (top-left), G (mid). Digits are updated via bitwise masks (`0x3F` for 0, `0x7F` for 8).

8.8 The glass bridge

The bridge is entirely boxes. Step i ($i = 0..15$) is at $z_i = 3 + 3.3\,i$, at height 16. The full bridge is 54.3 long (z from 2.0 to 56.3, centre 29.15).

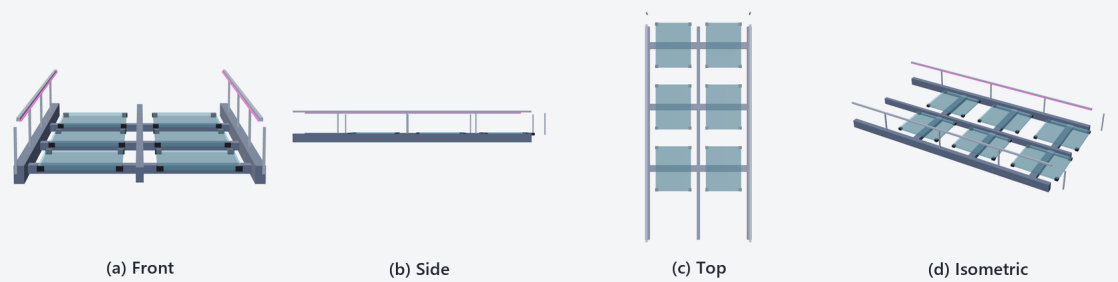


Figure 21: Glass bridge stepping stone multi-view projections: (a) Front, (b) Side profile, (c) Top view, (d) Isometric 3D view.

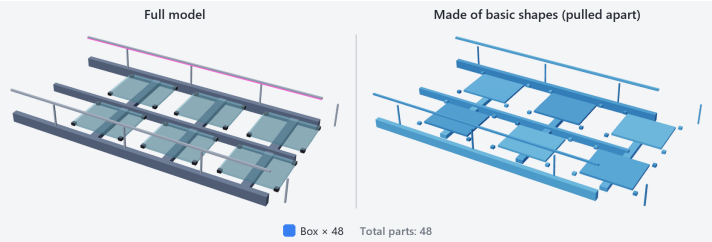


Figure 22: The first 3 steps of the bridge: 48 boxes (the full 16-step bridge uses 217).

Part	Shape	Scale / size	Translate (position)	Rotate
Steel beams $\times 2$	Box	$0.24 \times 0.42 \times 54.3$	$(\pm 2.75,\ 15.78,\ 29.15)$	–
Hand rails $\times 2$	Box	$0.08 \times 0.08 \times 54.3$	$(\pm 2.75,\ 17.05,\ 29.15)$	–
Pink neon strips $\times 2$	Box (glowing)	$0.03 \times 0.03 \times 54.3$	$(\pm 2.69,\ 17.02,\ 29.15)$	–
Rail posts $\times 34$	Box	$0.06 \times 1.05 \times 0.06$	$(\pm 2.75,\ 16.45,\ z_i - 1.65)$	–
Middle beam	Box	$0.18 \times 0.35 \times 54.3$	$(0,\ 15.78,\ 29.15)$	–
Cross beams $\times 16$	Box	$5.6 \times 0.20 \times 0.35$	$(0,\ 15.75,\ z_i)$	–
Glass panes $\times 32$	Box (42% see-through)	$1.90 \times 0.10 \times 2.40$	$(\pm 1.35,\ 15.98,\ z_i)$	–
Corner clamps $\times 128$	Box	$0.16 \times 0.12 \times 0.16$	pane $+(\pm 0.89,\ -0.06,\ \pm 1.13)$	–
“Safe” green glow	Box (glowing)	$1.82 \times 0.02 \times 2.30$	just above a proven pane	–
Broken pane marks	Box $\times 4$	$0.24 \times 0.02 \times 0.24$ (red)	at the four corners	–

Step pairs are generated along $+z$. Alpha blending ($\alpha = 0.42$) uses `glDepthMask(GL_FALSE)` during transparent rendering to maintain proper depth order.

8.9 Glass bridge start platform

Start platform placed with: $T(0,\ 15.75,\ -4.5)$. The finish platform is the same idea at the other end ($z = 61$), without rails. The deck uses the concrete texture.

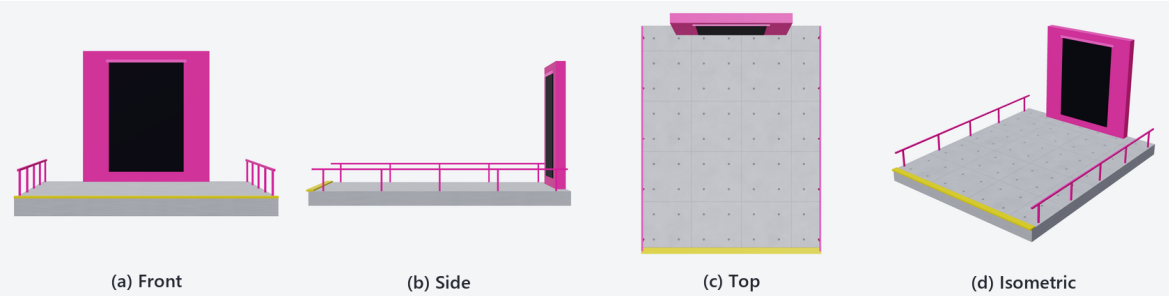


Figure 23: Glass bridge start platform multi-view projections: (a) Front, (b) Side profile, (c) Top view, (d) Isometric 3D view.

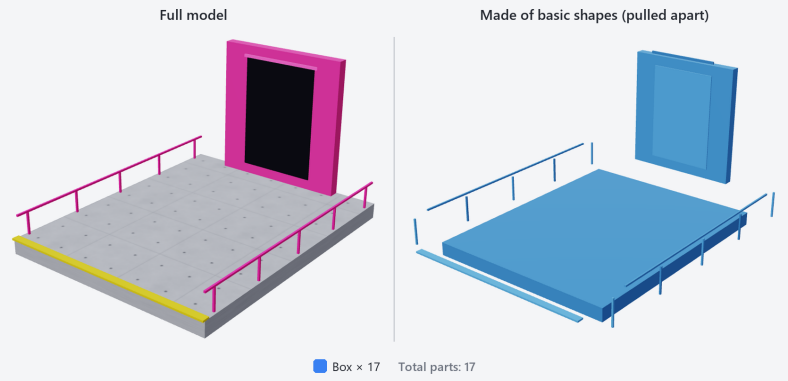


Figure 24: Start platform with its exit door: 17 boxes.

Part	Shape	Scale / size	Translate (position)	Rotate
Concrete deck	Box (textured)	$7.2 \times 0.5 \times 9.0$	$(0,\ 0,\ 0)$	–
Yellow edge	Box	$7.2 \times 0.06 \times 0.25$	$(0,\ 0.27,\ 4.45)$	–
Top rails $\times 2$	Box	$0.06 \times 0.06 \times 8.8$	$(\pm 3.5,\ 1.0,\ 0)$	–
Rail posts $\times 10$	Box	$0.06 \times 1.0 \times 0.06$	$(\pm 3.5,\ 0.5,\ -4, -2, 0, 2, 4)$	–
Door frame	Box	$4.4 \times 4.6 \times 0.35$	$(0,\ 2.5,\ -4.3)$	–
Dark doorway	Box	$2.6 \times 3.8 \times 0.42$	$(0,\ 2.5,\ -4.3)$	–
Glowing strip	Box (glowing)	$2.8 \times 0.10 \times 0.05$	door $+(0,\ 1.95,\ 0.22)$	–

The platform anchors contestants at height $y = 15.75$ prior to step 1. Bounding box collision restricts premature lateral movement before the round begins.

8.10 Round 2 bridge guard

Four guards stand watch on the elevated platforms, equipped with rifles and standard geometric uniform harnesses. **Placed with:** $T(x, 16, z) \cdot R_y(\text{angle})$: two guards at the start ($x = \pm 2.4$, angles $\pm 30^\circ$) and two at the finish.

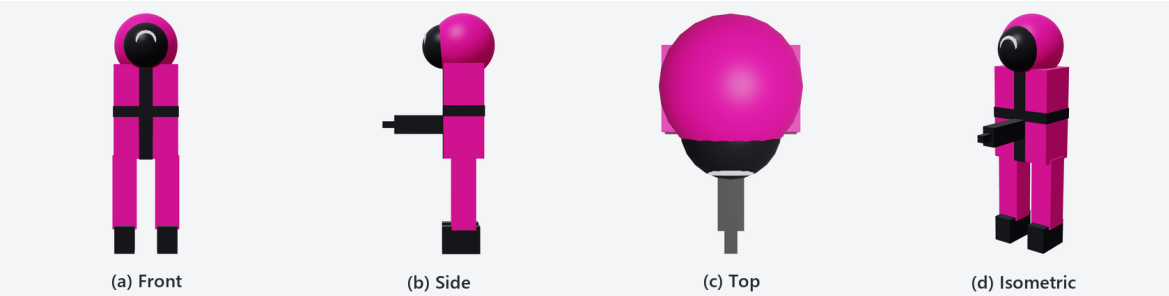


Figure 25: Round 2 bridge guard multi-view projections: (a) Front, (b) Side profile, (c) Top view, (d) Isometric 3D view.

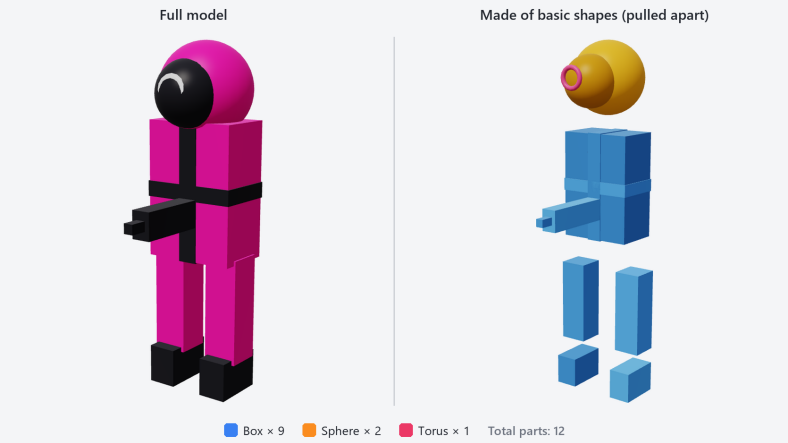


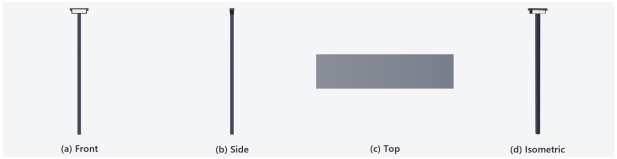
Figure 26: Bridge guard (simpler than the Round 1 guard): 9 boxes, 2 spheres, 1 torus.

Part	Shape	Scale / size	Translate (position)	Rotate
Boots $\times 2$	Box	$0.15 \times 0.20 \times 0.28$	$(\pm 0.16,\ 0.10,\ 0.02)$	–
Legs $\times 2$	Box	$0.18 \times 0.55 \times 0.18$	$(\pm 0.16,\ 0.45,\ 0)$	–
Body, belt, strap	Box $\times 3$	$0.48 \times 0.70 \times 0.30$; 0.49×0.08; 0.10×0.71	$(0,\ 1.05,\ 0)$	–
Rifle	Box $\times 2$	$0.10 \times 0.14 \times 0.52$; $0.05 \times 0.05 \times 0.70$	$(0,\ 0.95,\ 0.26)$	–
Head, mask	Sphere $\times 2$	radius 0.24; 0.17	$(0,\ 1.55,\ 0)$; $(0,\ 1.54,\ 0.14)$	–
Circle symbol	Torus	tube 0.015, ring 0.065	mask $+(0,\ 0.02,\ 0.14)$	–

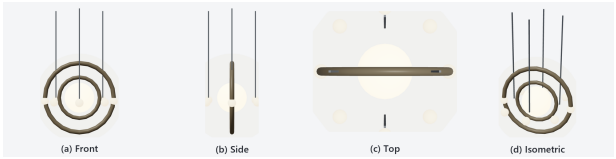
Bridge overseers feature circle rank emblems on their masks and carry low-poly rifles angled at $R_y \mp 30^\circ$ toward the contestants.

8.11 Light fixtures

Each light source in the simulation features a visible geometric housing rendering its physical presence.

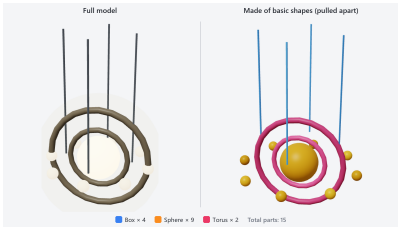


(a) Floodlight tower (Round 1)

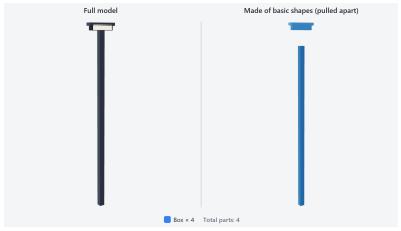


(b) Bridge chandelier (Round 2)

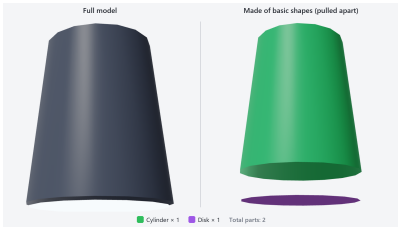
Figure 27: Light fixtures multi-view projections: (a) Front, (b) Side, (c) Top, (d) Isometric.



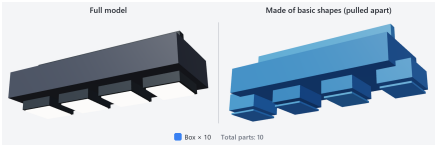
(a) Chandelier (point)



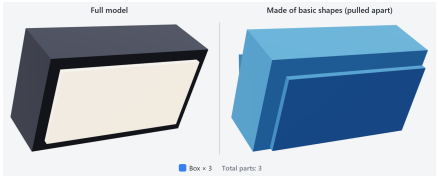
(b) Tower (point)



(c) Spotlight (spot)



(d) Gantry (directional)



(e) Wall lamp (point)

Figure 28: The light fixtures and their primitive shape composition.

Fixture	Parts (shape, size, translate, rotate)	Placed at
Floodlight tower	Box pole $0.45 \times 16 \times 0.45$ at $(0, 8, 0)$; Box bar $2.4 \times 0.25 \times 0.6$ at $(0, 16, 0)$; Box housing $2.0 \times 0.6 \times 0.5$ at $(0, 15.8, 0)$; glowing Box lamp $1.8 \times 0.45 \times 0.08$ at $(0, 15.75, -0.26)$	$T(18, 0, 15)$
Sun	glowing Sphere r 3.2 + 2 faint Sphere halos r 6.0, 10.5	$55 \cdot (\cos e \sin a,\ \sin e,\ \cos e \cos a)$, e=38°, a=45°
Chandelier	4 Box cables $0.06 \times 5.5 \times 0.06$ at $(1.5 \cos 90°k,\ 2.75,\ 1.5 \sin 90°k)$; 2 Torus rings (ring 2.2 and 1.3); 8 Sphere bulbs r 0.25 at $(2.2 \cos 45°i,\ -0.22,\ 2.2 \sin 45°i)$; glowing Sphere core r 0.95	$T(0, 23, 28)$
Follow spotlight	Cylinder body r 0.40 → 0.52, height 1.2, $R_x 90°$ (points down); glowing Disk lens r 0.5 at $(0, -1.22, 0)$	8.5 above the jumper
Ceiling gantry	Box rack $8.0 \times 1.2 \times 3.2$; Box hanger $7.2 \times 0.8 \times 0.4$ at $(0, 0.95, 0)$; 4 Box projectors $1.5 \times 0.8 \times 1.8$ at $(-3 + 2i,\ -0.6,\ 0)$, each with a glowing Box lens $1.35 \times 0.08 \times 1.6$	ceiling, y=28.2
Wall lamp ×4	Box bracket $0.25 \times 0.25 \times 1.5$ at $(0, 0, -0.75)$; Box housing $1.6 \times 0.9 \times 0.5$, $R_x 25°$; glowing Box face $1.4 \times 0.7 \times 0.05$	$(\pm 18.4,\ 23.5,\ 14$ or $42)$, $R_y \mp 90°$

Table 5: Light fixture parts.

8.12 Round 1 arena environment

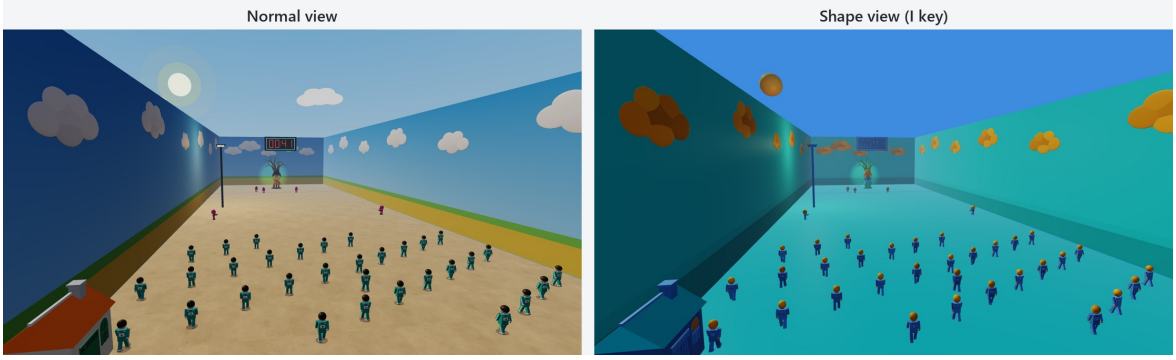


Figure 29: Round 1 arena, normal and exploded shape view. Sky dome and sand floor are polygons; clouds are clusters of spheres.

Part	Shape	Scale / size	Translate (position)	Rotate
Sky dome (new)	Polygon 32×10 quads	radius 190, pale horizon $\rightarrow$ deep blue top	centre $(0, 0, 0)$	–
Sky clouds (new) $\times 8$	5 Sphere each	squashed spheres, radius 6.5–9	150 away, 34–54 high	turned to face the arena
Sand floor	Polygon 24×32 quads (textured)	48×150, texture repeats every 6 units	$y = -0.05$	–
Painted sky walls $\times 4$	Polygon (quad)	48 or 150 long, 22 high, blue gradient	$x = \pm 24$, $z = \pm 75$	–
Wall clouds $\times 23$	6 Sphere each	$S(r,\ 0.65r,\ 0.08)$, $r = 1.7$–2.5	on the walls	–
Wheat and tree bands	Polygon $\times 8$	$y = 0$–3.2 and 3.2–4.2	just inside each wall	–
Start line	Box	$48 \times 0.01 \times 0.35$	$(0,\ 0.01,\ -52)$	–
Finish line	Box (glowing)	$48 \times 0.01 \times 0.45$	$(0,\ 0.01,\ 45)$	–

The arena boundary is enclosed by a hemisphere sky dome ($r = 190$) with smooth gradient vertex coloring and repeating sand tiles.

8.13 Round 2 glass bridge hall environment



Figure 30: Round 2 hall, normal and in exploded shape view.

Part	Shape	Scale / size	Translate (position)	Rotate
Side walls $\times 2$	Box (concrete)	$0.4 \times 48 \times 88$, texture 11×6 times	$(\pm 20,\ 10,\ 28)$	–
End walls $\times 2$	Box (concrete)	$40 \times 48 \times 0.4$	$(0,\ 10,\ -16)$, $(0,\ 10,\ 72)$	–
Ceiling, pit floor	Box (concrete)	$40 \times 0.4 \times 88$; $40 \times 0.5 \times 88$	$y = 34$; $y = -14$	–
Ceiling light strips $\times 2$	Box $\times 2$ each	$1.6 \times 0.25 \times 82$ + glowing panel	$(\pm 8,\ 33.7,\ 28)$	–
Wall ribs + LED bars $\times 22$	Box $\times 2$ each	$0.22 \times 48 \times 0.40$; $0.04 \times 6 \times 0.10$	$x = \pm 19.76$, $z = -9, -1, \ldots, 71$	–
Neon symbols $\times 6$	Torus or 3/4 Box + backing Box	ring radius 2.0; triangle side 4.2; square side 3.6	$(\pm 19.7,\ 25,\ 9 \mid 29 \mid 49)$	$R_y \mp 90°$; triangle sides $R_z \pm 30°$
Neon wall lines $\times 2$	Box (glowing)	$0.05 \times 0.14 \times 88$	$(\pm 19.78,\ 16,\ 28)$	–
Roof trusses $\times 5$	Box	$40 \times 0.6 \times 0.6$	$(0,\ 28.5,\ 0..56$ every $14)$	–
Hanging cables $\times 10$	Cylinder	radius 0.025, length 11.4	$(\pm 2.75,\ 17.1,$ truss $z)$	$R_x -90°$
Safety net	Box $\times 39$	cables 0.03 thick	$y = 2$, 14×60 area	–

The hall features 22 pairs of LED wall ribs, structural roof trusses, hanging suspension cables, and an emergency safety net spanning 14×60 units.

9 Lighting

The game uses all three classic light types. The shader adds up the light from every lamp for every pixel.

Light type	Idea	Round 1	Round 2
Directional	Far away like the sun. Same direction everywhere, no fading.	The sun (elevation $38°$, azimuth $45°$)	Ceiling gantry (elevation $50°$)
Point	Shines in all directions from one spot. Gets weaker with distance.	Floodlight tower at $(18, 16, 15)$	Chandelier at $(0, 23, 28)$ **+ 4 new wall lamps**
Spot	A point light limited to a cone, with a soft edge.	From the doll's eyes: red when she looks, green when she sings	Follows the jumping player from above

Table 6: The three light types in each round. Keys `0`, `8`, `9` turn them on and off.

How strong is the light at a point?

- **Diffuse** (matte brightness): $\max(N \cdot L,\ 0)$ – brightest when the surface faces the light.
- **Specular** (shiny spot, Blinn-Phong): $\max(N \cdot H,\ 0)^n$, where $H = \dfrac{L+V}{|L+V|}$ and n is the shininess (bigger n = smaller, sharper highlight).
- **Point light fading:** $f(d) = \dfrac{I}{1 + k_l\, d + k_q\, d^2}$ (tower: $I{=}1.15,\ k_l{=}0.035,\ k_q{=}0.005$; wall lamps: $I{=}1.25,\ k_l{=}0.030,\ k_q{=}0.004$).
- **Spot cone:** $\text{cone} = \text{clamp}\left(\dfrac{\cos\theta - \cos 23°}{\cos 15° - \cos 23°},\ 0,\ 1\right)$. Inside $15°$ the light is full, outside $23°$ it is zero, in between it fades.

```
// GLSL Point Light computation with quadratic distance attenuation
vec3 calcPointLight(PointLight light, vec3 normal, vec3 fragPos, vec3 viewDir) {
   vec3 lightDir = normalize(light.position - fragPos);
   float diff = max(dot(normal, lightDir), 0.0);
   vec3 halfwayDir = normalize(lightDir + viewDir);
   float spec = pow(max(dot(normal, halfwayDir), 0.0), shininess);
   float dist = length(light.position - fragPos);
   float atten = 1.0 / (light.constant + light.linear * dist + light.quadratic * dist * dist);
   return (light.ambient + diff * light.diffuse + spec * light.specular) * atten;
}
```

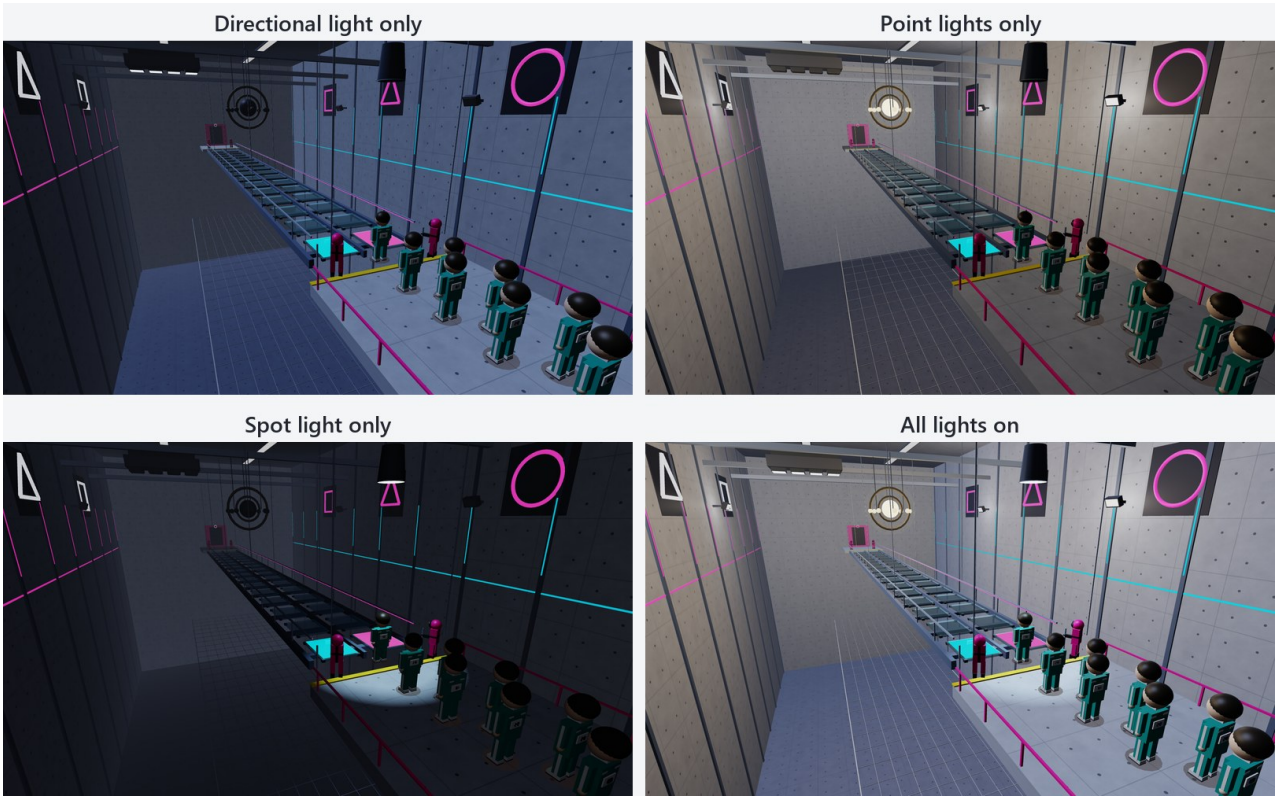


Figure 31: Round 2. Directional: even light from above. Point: warm light around the chandelier and the wall lamps. Spot: a bright circle on the player waiting at the front.

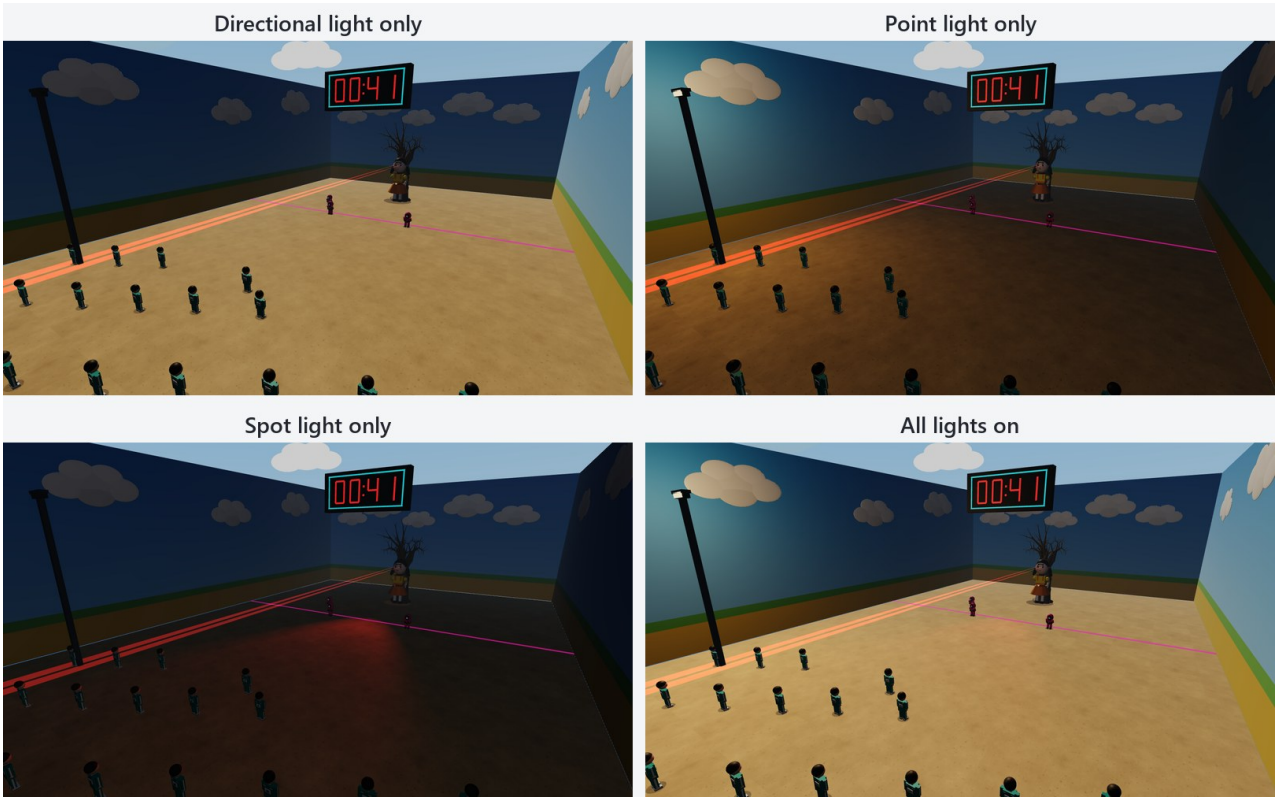


Figure 32: Round 1 (red light). Directional: sunlight on everything. Point: warm glow near the tower. Spot: a red cone from the doll's eyes lights the sand in front of her.

Extra light effects.

- **Ambient:** a soft base light so shadows are not pitch black. It mixes a sky color (from above) and a ground color (from below).
- **Emission:** glowing parts (neon, lamp faces, the clock digits) add their own color.
- **Haze:** far away surfaces slowly fade into a haze color, visibility $= e^{-(0.0075\,d)^2}$ in Round 2. This adds depth to big spaces.

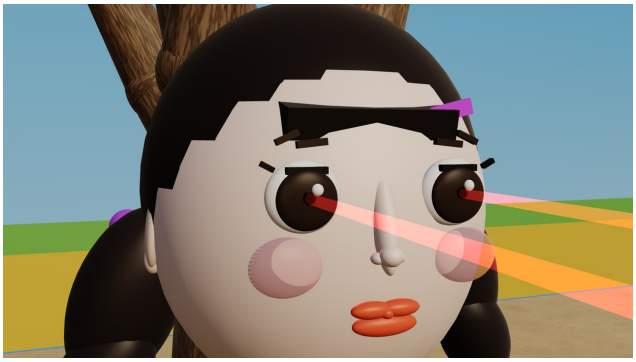
10 Phong Shading

Shading decides the color of each pixel inside a triangle. This project uses **Phong shading**: the light is calculated *separately for every pixel*.

1. The **vertex shader** only moves each corner (vertex) into place and passes its normal on.
2. The GPU **interpolates** (blends) the normals across the triangle.
3. The **fragment shader re-normalizes** the normal for each pixel and computes ambient + diffuse + specular for every light.

$$\text{color} = \underbrace{k_a\, C_{\text{ambient}}}_{\text{ambient}} + \sum_{\text{lights}} \Big(\underbrace{\max(N\cdot L, 0)\, C_{\text{light}}}_{\text{diffuse}} \text{ base} + \underbrace{k_s \max(N\cdot H, 0)^n\, C_{\text{light}}}_{\text{specular}} \Big) + \text{emission}$$

Because the normal is correct at every pixel, curved things look smooth and get small, sharp highlights even in the middle of a triangle – see the doll's eyes and lips and the players' heads.



(a) Shiny highlights on the doll's eyes, lips and hair.



(b) Smooth light and highlights on round heads.

Figure 33: Phong shading, close up.

The heart of the fragment shader (`src/shaders/default.frag`):

```
// Blinn-Phong diffuse + specular of one light coming from direction L
void addLight(vec3 N, vec3 V, vec3 L, vec3 lightColor, float diffuseScale,
              float specScale, float specExp, inout vec3 diffuse, inout vec3 specular) {
    float diff = max(dot(N, L), 0.0); // Lambert: cos(angle to light)
    diffuse += diff * lightColor * diffuseScale;
    vec3 H = normalize(L + V); // halfway vector
    float spec = pow(max(dot(N, H), 0.0), specExp); // shininess = highlight size
    specular += spec * lightColor * specScale * materialSpecularStrength;
}
...
vec3 N = normalize(Normal); // per-pixel normal: this is Phong shading
vec3 V = normalize(viewPos - FragPos); // direction to the camera
```

11 Texture Mapping

A **texture** is an image wrapped onto a surface. This version adds three textures (on purpose, not too many). They were made by a small script (`tools/make_textures.py`) so they tile without seams.



Figure 34: The three texture images (512×512 PNG).

How it works.

1. Each vertex now has a **texture coordinate** (u, v) between 0 and 1 that says which point of the image belongs to it. Boxes get a $(0,0)$–$(1,1)$ square on each face, cylinders wrap u around and run v along the length, spheres use longitude/latitude.
2. The image is loaded with `stb_image` and uploaded once as an OpenGL texture with mipmaps and `GL_REPEAT`.
3. In the fragment shader: base $=$ material $\times$ texture$(u \cdot r_u,\ v \cdot r_v)$. The repeat counts (r_u, r_v) tile the image so it keeps a real size on big surfaces.

Texture	Used on	How it repeats
Sand	Round 1 floor (48×150)	$u = x/6,\ v = z/6$: one image per 6×6 units
Bark	Every branch of the dead tree	around: $\approx 4\times$radius times; along: length$/1.6$ times
Concrete	Round 2 walls, ceiling, pit floor and both platforms	one image per 8×8 units (e.g. side wall 11×6)

```
GLCompat::setTexture(&SceneTextures::concrete(), 88.0f / 8.0f, 48.0f / 8.0f); // 11 x 6 repeats
GLCompat::drawBox(0.4f, 48.0f, 88.0f); // side wall
GLCompat::clearTexture();
// fragment shader:
if (useTexture) baseColor *= texture(diffuseTexture, TexCoord * texRepeat).rgb;
```



(a) Sand floor **(b)** Bark on the tree **(c)** Concrete hall wall

Figure 35: The textures inside the game.

12 Animation and Game Logic

What moves	How	Formula
Walking legs and arms	Legs swing forward and back like a pendulum; arms swing the other way.	$\theta_{\text{leg}} = 28^\circ \sin(10\,t)$, $\theta_{\text{arm}} = -\theta_{\text{leg}}$
Doll's head	Turns smoothly to the target angle (0° = singing, 180° = looking).	angle $+=$ (target $-$ angle) $\cdot\, 15\,\Delta t$
Glass bridge jump	Moves straight to the next pane and rises in an arc.	$y = 16 + 1.3\sin(\pi p)$, $p : 0 \to 1$ in 0.42 s
Falling player	Speed grows every frame (gravity).	$v \mathrel{+}= 32\,\Delta t$, $y \mathrel{-}= v\,\Delta t$
Glass shards (55)	Random start speed and spin, then gravity.	$v_y \mathrel{-}= 28\,\Delta t$; $\vec{p} \mathrel{+}= \vec{v}\,\Delta t$
Blood drops (30)	Burst outward, fall, fade out.	$v_y \mathrel{-}= 9.8\,\Delta t$; alpha $= 1 -$ life/max

Round 1 rules.

- 60 seconds to reach the finish line ($z = 45$).
- Green light: the doll sings for 4.8–6.2 seconds. Red light: she looks for 3.2–5.2 seconds.
- If Player 456 moves while her head is turned past 80°, a guard shoots and the game is over.
- Each red light, every other contestant has an 18% chance to be caught.

Round 2 rules.

- 16 players take turns. Player 456 gets a random place in the queue.
- For every step, the strong side is chosen at random (50/50). Once a pane is proven, later players follow it.
- The other players choose randomly. Player 456 chooses with `A` / `D`. 16 minutes in total.

13 Cameras and Viewport Pipeline

The project implements a modern dual-layer camera system combining the core engine `Camera` class in `src/engine/Camera` and the high-level game director `CameraManager` in `src/camera/Camera.cpp`. It supports 7 real-time cinematic perspectives, interactive 360° mouse orbiting, dynamic scroll zoom, exponential smoothing, and multi-pass Picture-in-Picture (PiP) rendering.

13.1 Camera Architecture and Mathematical Mechanism

The camera pipeline converts 3D world-space coordinates into 2D screen coordinates using the standard Model-View-Projection (MVP) matrix transformation:

$$\vec{v}_{\text{clip}} = P \cdot V \cdot M \cdot \vec{v}_{\text{local}} \tag{1}$$

1. View Matrix (V): Derived from an eye position $\vec{P}_{\text{eye}}$, a look-at target $\vec{P}_{\text{target}}$, and a world up vector $\vec{U} = (0, 1, 0)$ via `glm::lookAt`:

$$\vec{F} = \frac{\vec{P}_{\text{target}} - \vec{P}_{\text{eye}}}{\|\vec{P}_{\text{target}} - \vec{P}_{\text{eye}}\|}, \quad \vec{R} = \frac{\vec{F} \times \vec{U}}{\|\vec{F} \times \vec{U}\|}, \quad \vec{U}' = \vec{R} \times \vec{F} \tag{2}$$

$$V = \begin{bmatrix} R_x & R_y & R_z & -\vec{R} \cdot \vec{P}_{\text{eye}} \\ U'_x & U'_y & U'_z & -\vec{U}' \cdot \vec{P}_{\text{eye}} \\ -F_x & -F_y & -F_z & \vec{F} \cdot \vec{P}_{\text{eye}} \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{3}$$

2. Perspective Projection Matrix (P): Defined by the vertical field of view (fov_y), viewport aspect ratio ($\text{aspect} = W/H$), near clipping plane ($z_n = 0.1$), and far clipping plane ($z_f = 280.0$):

$$P = \begin{bmatrix} \frac{1}{\text{aspect} \cdot \tan(\text{fov}/2)} & 0 & 0 & 0 \\ 0 & \frac{1}{\tan(\text{fov}/2)} & 0 & 0 \\ 0 & 0 & -\frac{z_f + z_n}{z_f - z_n} & -\frac{2 z_f z_n}{z_f - z_n} \\ 0 & 0 & -1 & 0 \end{bmatrix} \tag{4}$$

The core camera matrix update implemented in `src/engine/Camera.cpp`:

```
void Camera::SetView(const glm::vec3& eye, const glm::vec3& target, const glm::vec3& up,
                     float fovDeg, float nearPlane, float farPlane) {
    Position = eye;
    Orientation = glm::normalize(target - eye);
    Up = up;
    view = glm::lookAt(eye, target, up);
    projection = glm::perspective(glm::radians(fovDeg),
                                  (float)width / (float)height,
                                  nearPlane, farPlane);
}
```

13.2 Screen Dimensions, Aspect Ratio, and Viewport Calculation

A fundamental requirement in 3D graphics is ensuring that objects retain their true geometric proportions regardless of screen resolution or window resizing. Without continuous aspect ratio recalculation, spheres and character heads would distort into stretched ellipsoids.

Resolution Handling and GLFW Callbacks: The game defaults to a 1600×900 desktop window and scales seamlessly to Full HD 1920×1080, 2560×1440, or user-resized dimensions. Physical framebuffer dimensions are tracked via GLFW:

```
void framebufferSizeCallback(GLFWwindow*, int w, int h) {
    windowWidth = (w > 0) ? w : 1;
    windowHeight = (h > 0) ? h : 1;
}
```

Split-Screen HUD Viewport Calculation: The game UI features two persistent sidebar panels:

- **Left Sidebar ($W_{\text{left}} = 300$ px):** Houses the live CCTV doll camera feed, countdown timer, contestant counters, and arena mini-radar.
- **Right Sidebar ($W_{\text{right}} = 300$ px):** Houses camera mode buttons, speed controls, and interactive lighting toggles.

Rather than rendering the 3D scene underneath the HUD overlays, the 3D rendering pipeline calculates an independent center viewport:

$$W_{\text{center}} = W_{\text{window}} - W_{\text{left}} - W_{\text{right}}, \qquad \text{Aspect}_{\text{3D}} = \frac{W_{\text{center}}}{H_{\text{window}}} \tag{5}$$

For example, on a standard Full HD display of 1920×1080:

- Total screen: $W_{\text{window}} = 1920$, $H_{\text{window}} = 1080$. Fullscreen aspect ratio: $\frac{1920}{1080} = \frac{16}{9} \approx 1.778$.
- Center 3D viewport: $W_{\text{center}} = 1920 - 300 - 300 = 1320$ px.
- Rendered 3D aspect ratio: $\text{Aspect}_{\text{3D}} = \frac{1320}{1080} \approx 1.222$.

The viewport and projection matrices are dynamically updated every frame in `src/main_modern.cpp`:

```
int leftBarW = UI_LEFT_BAR_WIDTH, rightBarW = UI_RIGHT_BAR_WIDTH;
int centerW = (drawUI && windowWidth > 600) ? (windowWidth - leftBarW - rightBarW) : windowWidth
    ;
glViewport(drawUI ? leftBarW : 0, 0, centerW, windowHeight);
camera.width = centerW; camera.height = windowHeight;
camera.SetView(glm::vec3(eyeX, eyeY, eyeZ), glm::vec3(lookX, lookY, lookZ),
            glm::vec3(upX, upY, upZ), fov, 0.1f, 280.0f);
```

When the user toggles the HUD off (`H`), the sidebars collapse to 0, expanding $W_{\text{center}} = 1920$ and restoring the native $16:9$ (1.778) aspect ratio without any geometric stretching.

13.3 Smooth Camera Easing and Interactive Orbiting

To prevent abrupt camera snapping when switching perspectives or moving rapidly, the camera interpolates its coordinates using exponential damping:

$$\vec{P}_{\text{cam}}(t + \Delta t) = \vec{P}_{\text{cam}}(t) + \left(\vec{P}_{\text{target}} - \vec{P}_{\text{cam}}(t)\right) \cdot \left(1 - e^{-k\,\Delta t}\right) \tag{6}$$

where $k = 12.0$ for close-up tracking and $k = 8.0$ for wide arena perspectives.

Mouse Orbit Controls: Users can freely orbit 360° around the player or the doll by dragging the mouse. The pitch (θ) and yaw (ϕ) spherical coordinates are translated into Cartesian eye offsets:

$$\Delta x = d\cos\theta\sin\phi, \quad \Delta y = d\sin\theta, \quad \Delta z = d\cos\theta\cos\phi \tag{7}$$

Mouse scroll wheel deltas dynamically scale distance d, clamped within safety limits ($1.8 \le d \le 18.0$ for 3rd person; $2.0 \le d \le 38.0$ for the doll).

13.4 Camera Perspectives and Multi-Pass CCTV Rendering

The game features 7 primary camera perspectives and an independent secondary rendering pass:

Key	Camera Mode	FOV	Behavior and Target
1	**3rd Person Follow**	55°	Follows Player 456; supports 360° mouse orbit and arena collision clamping
2	**Overhead Drone**	55°	High stadium overview ($d = 58$); full arena free-orbit and zoom
3	**Top-Down Aerial**	58°	Plan view centered at $(0, 110, -3.5)$ with north aligned forward
4	**Doll 360° Orbit**	55°	Orbiting camera anchored at the doll's head $(0, 8.5, 58)$
5	**Left Broadcast**	55°	Elevated broadcast perspective from the left wall $(-20.5, 12, z_{\text{player}} + 5)$
6	**Right Broadcast**	55°	Symmetrical broadcast perspective from the right wall $(+20.5, 12, z_{\text{player}} + 5)$
7	**Finish Line POV**	55°	Located at $z = 45$ facing backwards to watch oncoming contestants

Table 7: The 7 dynamic camera modes available in the game.

Multi-Pass CCTV Doll Camera: In addition to the main viewport, the engine executes a dedicated secondary rendering pass in Round 1:

1. **Pass 1 (Main Scene):** Renders the central 3D viewport using the player's active camera mode.
2. **Pass 2 (HUD):** Draws ImGui sidebars and panels over the window.
3. **Pass 3 (CCTV Doll Cam):** Renders an over-the-shoulder POV (FOV $= 48^\circ$) into the left panel frame via `glViewport`.

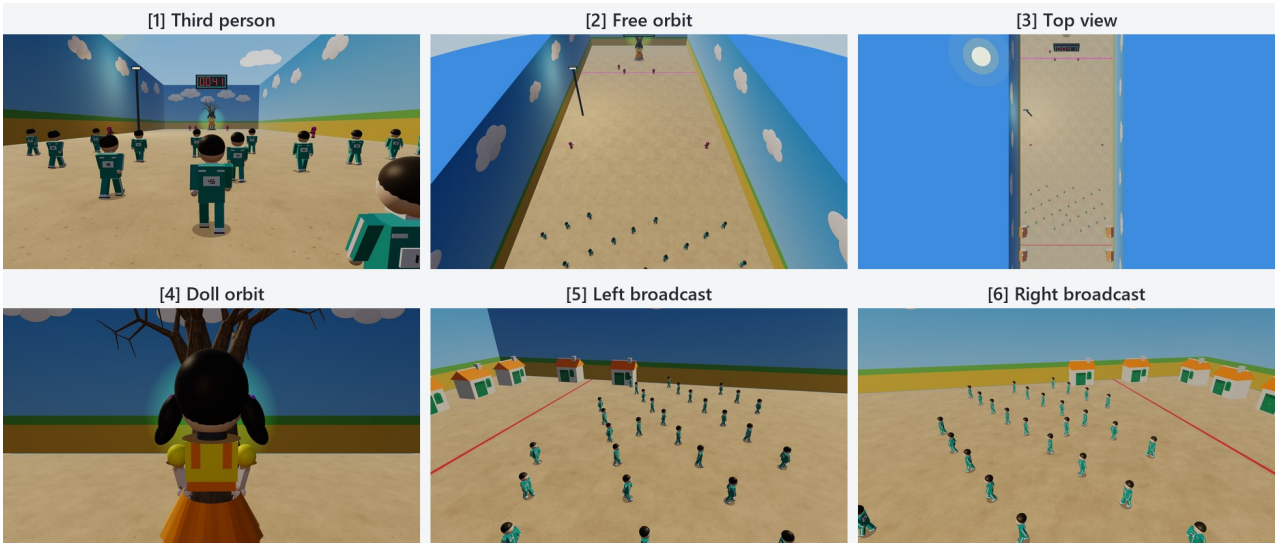


Figure 36: Round 1 camera modes (keys 1–6).

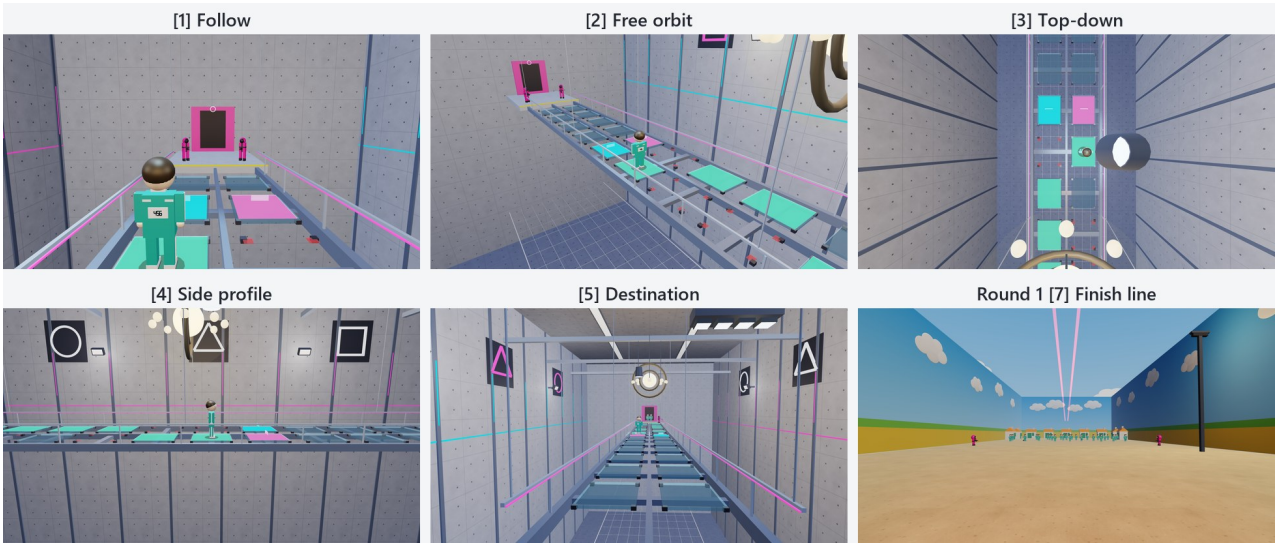


Figure 37: Round 2 camera modes (keys 1–5) and Round 1's finish-line camera (key 7, during red light).

14 Conclusion

This project builds a complete two-round 3D game without loading a single 3D model. Seven basic shapes, placed with translate, rotate and scale and organised in parent–child groups, make every object – from a contestant (53 parts) to the giant doll (147 parts) and the recursive tree (313 parts). The scenes are lit by directional, point and spot lights, shaded per pixel with Phong shading, and three textures add surface detail. The new capture and shape-view tools make it easy to see, check and explain how everything is built.